

Algorithms for Computing Variants of the Longest Common Subsequence Problem

Extended Abstract

M. Sohel Rahman^{*,**} and Costas S. Iliopoulos^{***}

Algorithm Design Group
Department of Computer Science, King's College London,
Strand, London WC2R 2LS, England
{sohel, csi}@dcs.kcl.ac.uk
<http://www.dcs.kcl.ac.uk/adg>

Abstract. The *longest common subsequence*(LCS) problem is one of the classical and well-studied problems in computer science. The computation of the LCS is a frequent task in DNA sequence analysis, and has applications to genetics and molecular biology. In this paper we define new variants, introducing the notion of gap-constraints in LCS problem and present efficient algorithms to solve them.

1 Introduction

The *longest common subsequence*(LCS) problem is one of the classical and well-studied problems in computer science which has extensive applications in diverse areas ranging from spelling error corrections to molecular biology [7, 2]. This paper introduces the notion of gap-constraints in LCS. Our versions of LCS, on one hand, offers the possibility to handle gap-constraints between the consecutive matches among the sequences. On the other hand, they provide us with the tool to handle motif finding problems where not all positions of the motif is important [6]. Before going into details we need to present some preliminary concepts.

Suppose we are given two strings $X[1..n] = X[1] X[2] \dots X[n]$ and $Y[1..n] = Y[1] Y[2] \dots Y[n]$. A subsequence $S[1..r] = S[1] S[2] \dots S[r]$ of X is obtained by deleting $n - r$ symbols from X . A common subsequence of two strings X and Y , denoted $cs(X, Y)$, is a subsequence common to both X and Y . The longest common subsequence of X and Y , denoted $lcs(X, Y)$ or $LCS(X, Y)$, is a common subsequence of maximum length. We denote the length of $lcs(X, Y)$ by $r(X, Y)$.

Problem “LCS”. *Given 2 strings X and Y , we want to find out the Longest Common Subsequence of X and Y .*

* Supported by the Commonwealth Scholarship Commission in the UK under the Commonwealth Scholarship and Fellowship Plan (CSFP).

** On Leave from Department of CSE, BUET, Dhaka-1000, Bangladesh.

*** Supported by EPSRC and Royal Society grants.

In what follows we assume that the two given strings are of equal length. But our results can be easily extended to handle two strings of different length.

Definition. (Correspondence Sequence): *Given a string $X[1..n]$ and a subsequence $S[1..r]$ of X , we define a correspondence sequence (not necessarily unique) of X and S , $C(X, S) = C[1] C[2] \dots C[r]$ to be a strictly increasing sequence of integers taken from $[1, n]$ such that $S[i] = X[C[i]]$ for all $1 \leq i \leq r$.*

Definition. (Fixed Gapped Correspondence Sequence): *A correspondence sequence of a string X of length n and one of its subsequences S of length r is said to be a Fixed Gapped Correspondence Sequence with respect to a given integer K if and only if we have $C[i] - C[i - 1] \leq K + 1$ for all $2 \leq i \leq r$. We sometimes use $C_{FG(K)}$ to denote a Fixed Gapped Correspondence Sequence with respect to K .*

Definition. (Elastic Gapped Correspondence Sequence): *A correspondence sequence of a string X of length n and one of its subsequences S of length r is said to be a Elastic Gapped Correspondence Sequence with respect to given integers K_1 and K_2 , $K_2 > K_1$, if and only if we have $K_1 < C[i] - C[i - 1] \leq K_2 + 1$ for all $2 \leq i \leq r$.*

Definition. (Fixed and Elastic Gapped Common Subsequence): *Suppose we are given two strings $X[1..n]$ and $Y[1..n]$ and an integer K . A common subsequence $S[1..r]$ of X and Y is a Fixed Gapped Common Subsequence, if there exists Fixed Gapped Correspondence Sequences $C_{FG(K)}(X, S)$ and $C_{FG(K)}(Y, S)$. For a Fixed Gapped Common Subsequence to be **rigid** we must also have, for all $2 \leq i \leq r$, $C_{FG(K)}(X, S)[i] - C_{FG(K)}(X, S)[i - 1] = C_{FG(K)}(Y, S)[i] - C_{FG(K)}(Y, S)[i - 1]$. Elastic Gapped Common Subsequences (both non-rigid and rigid) can be defined analogously.*

The problems we handle in this paper are defined in Figure 1. In this paper we use the following notions. We say a pair (i, j) , $1 \leq i, j \leq n$ defines a match, if $X[i] = Y[j]$. The set of all matches, M , is defined as follows: $M = \{(i, j) \mid X[i] = Y[j], 1 \leq i, j \leq n\}$. We define $|M| = \mathcal{R}$.

PROBLEM	INPUT	OUTPUT (of maximum length)
FIG (Fixed Gap)	X, Y and K	Fixed Gapped Common Subsequence
ELAG (Elastic Gap)	X, Y, K_1 and K_2	Elastic Gapped Common Subsequence
RIFIG (Rigid Fixed Gap)	X, Y and K	Rigid Fixed Gapped Common Subsequence
RELAG (Rigid Elastic Gap)	X, Y, K_1 and K_2	Rigid Elastic Gapped Common Subsequence

Fig. 1. Problems handled in this paper

2 An Algorithm for FIG

In the traditional dynamic programming technique to solve LCS [11], the idea is to determine the longest common subsequences for all possible prefix combina-

tions of the input strings. The recurrence relation for extending $r(X[1..i], Y[1..j])$, is as follows [11]:

$$\mathcal{T}[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \mathcal{T}[i - 1, j - 1] + 1 & \text{if } X[i] = Y[j], \\ \max(\mathcal{T}[i - 1, j], \mathcal{T}[i, j - 1]) & \text{if } X[i] \neq Y[j]. \end{cases} \quad (1)$$

Here we have used the tabular notion $\mathcal{T}[i, j]$ to denote $r(X[1..i], Y[1..j])$. After the table has been filled, $r(X, Y)$ can be found in $\mathcal{T}[n, n]$ and $lcs(X, Y)$ can be found by backtracking from $\mathcal{T}[n, n]$. Unfortunately, the attempt to generalize this algorithm in a straightforward way doesn't give us an algorithm to solve our problems. Note that, in FIG, due to the gap constraint, a continuing common sequence may have to stop at an arbitrary $\mathcal{T}[i, j]$ because the next match is not within the gap constraint. In order to cope with this situation what we do is as follows. For each tabular entry $\mathcal{T}[i, j], (i, j) \in M$ we calculate and store two values namely $\mathcal{T}_{local}[i, j]$ and $\mathcal{T}_{global}[i, j]$. For all other $(i, j), \mathcal{T}_{local}[i, j]$ is irrelevant and, hence, is undefined. The recurrence relations are defined below:

$$\mathcal{T}_{local}[i, j] = \begin{cases} \text{Undefined} & \text{if } (i, j) \notin M, \\ \max_{\substack{i-1-K \leq \ell_i < i \\ j-1-K \leq \ell_j < j \\ (\ell_i, \ell_j) \in M}} (\mathcal{T}_{local}[\ell_i, \ell_j]) + 1 & \text{if } (i, j) \in M. \end{cases} \quad (2)$$

Remark 1. The max operation in Equation 2 returns 0, when there is no $(\ell_i, \ell_j) \in M, i - 1 - K \leq \ell_i < i, j - 1 - K \leq \ell_j < j$.

$$\mathcal{T}_{global}[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1]) & \text{if } (i, j) \notin M, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1], \mathcal{T}_{local}[i, j]) & \text{if } (i, j) \in M. \end{cases} \quad (3)$$

It is easy to see that $\mathcal{T}_{global}[i, j]$ is used to store the information of the LCS so far, i.e. the 'global' LCS, on other hand $\mathcal{T}_{local}[i, j]$ tracks any 'local' LCS in growth. As soon as a local LCS becomes the global one the value of the corresponding $\mathcal{T}_{global}$ changes. What will be the running time of this algorithm? Since, for each $(i, j) \in M$, we have to check a $(K + 1)^2$ area to find the maximum of $\mathcal{T}_{local}$ in that area, the total running time is $O(n^2 + \mathcal{R}(K + 1)^2)$. The space requirement is $\theta(n^2)$. Also note that by keeping appropriate pointer information or by examining the calculation of the two recurrences we can easily construct $lcs(X, Y)$ as can be done in the case of the traditional solution of LCS problem.

Theorem 1. *Problem FIG can be solved in $O(n^2 + \mathcal{R}(K + 1)^2)$ time using $\theta(n^2)$ space.* $\square$

Remark 2. Unfortunately, this strategy, if used for the LCS problem, would lead to an inefficient algorithm with $O(n^2 + \sum_{(i,j) \in M} (i - 1)(j - 1))$ worst case running time.

As in the case of the traditional solution of LCS problem the complete tabular information is required for our algorithm, basically, to provide the solution for the subproblems if required. If that is not required, we can get rid of $\mathcal{T}_{global}$ altogether by keeping a variable to keep track of the current global LCS. As

soon as a local LCS becomes a global one we just change this variable. This would give us a running time of $O(\mathcal{R}(K+1)^2)$ provided we have a preprocessing step to construct the set M in sorted order according to their position they would be considered in the algorithm. This preprocessing step is as follows. We construct for each symbol $a \in \Sigma$ two separate lists, $L_X[a]$ and $L_Y[a]$. For each $a \in \Sigma$, $L_X[a]$ ($L_Y[a]$) stores, in sorted order, the positions of a in X (Y), if any. We now use an elegant data structure invented by Emde Boas [10] that allows us to maintain a sorted list of integers in the range $[1..n]$ in $O(\log \log n)$ time per insertion and deletion. In addition to that it can return $next(i)$ (successor element of i in the list) and $prev(i)$ (predecessor element of i in the list) in constant time. We construct an Emde Boas data structure E_P where we insert each pair $(i, j), i \in L_X[a], j \in L_Y[a], a \in \Sigma$. In this case the order of the elements in E_P is maintained according to the value $(i * (n-1) + j)$. Note that these values are within the range $[1..n^2]$ and hence the cost is $O(\log \log n^2) = O(\log \log n)$ per insertion and deletion. It is easy to verify that, using E_P , we can get all the pairs in the correct order to process them in row by row manner. We now analyze the running time of this preprocessing step. The $2 * |\Sigma|$ lists can be constructed in $O(n)$ by simply scanning X and Y in turn. Since there are in total $\mathcal{R}$ elements in M , the construction of E_P requires $O(\mathcal{R} \log \log n)$ time. The space requirement is $O(\mathcal{R})$. We note, however, that for the complete algorithm, we still need the $\theta(n^2)$ space to guaranty a linear time search for the highest $\mathcal{T}_{local}$ in the $(K+1)^2$ area.

Theorem 2. *Given a preprocessing time of $O(\mathcal{R} \log \log n)$, Problem FIG can be solved in $O(\mathcal{R}(K+1)^2)$ time.* $\square$

3 An Improved Algorithm for FIG

In this section we try to improve the running time of Algorithm presented in Section 2. Ideally, we would like to reduce the quadratic term $(K+1)^2$ to linear. Note that we can easily improve the running time of the algorithm to LCS problem reported in Remark 2 using the following interesting facts.

Fact 1. *Suppose $(i, j) \in M$. Then for all $(i', j), i' > i$ ($(i, j'), j' > j$), we must have $\mathcal{T}[i', j] \geq \mathcal{T}[i, j]$ ($\mathcal{T}[i, j'] \geq \mathcal{T}[i, j]$), where $\mathcal{T}$ is the table filled up by the traditional dynamic programming algorithm using Equation 1.* $\square$

Fact 2. *The calculation of a $\mathcal{T}[i, j], (i, j) \in M, 1 \leq i, j \leq n$ is independent of any $\mathcal{T}[\ell, q], (\ell, q) \in M, \ell = i, 1 \leq q \leq n$.* $\square$

The idea is to avoid checking the $(i-1)(j-1)$ entries and check only $(j-1)$ (or $(i-1)$) entries instead. We maintain an array H of length n where, for $\mathcal{T}[i, j]$ we have, $H[\ell] = \max_{1 \leq k < i, (i, \ell) \in M} (\mathcal{T}[k, \ell]), 1 \leq \ell \leq n$. The ‘max’ operation, here, returns 0 if there exists no $(i, \ell) \in M$ within the range. Given the updated array H , we can easily perform the task by checking only the $(j-1)$ entries of H . And Fact 1 makes it easy to maintain the array H on the fly as we proceed as follows. As usual, we proceed in a row by row manner. We use another array S , of length

n , as a temporary storage. When we find an $(i, j) \in M$, after calculating $\mathcal{T}[i, j]$ we store $S[j] = \mathcal{T}[i, j]$. We continue to store in this way as long as we are in the same row. As soon as we find an $(i', j) \in M, i' > i$, i.e. we start processing a new row, we update H with new values from S .

The correctness of the above procedure¹ follows from Fact 1 and 2. But this idea doesn't work for FIG because Fact 1 doesn't hold when we consider FIG. This is because due to the gap constraint a new local LCS may start which would surely have lesser $\mathcal{T}$ -value than another previous local LCS. We, however, use the similar idea to improve our previous running time. But we need to do something more than just maintaining an array. In the rest of this section we present a novel technique to present the improved algorithm. The basic idea depends on the following fact which is, basically, an extension of Fact 2.

Fact 3. *The calculation of a $\mathcal{T}_{local}[i, j], (i, j) \in M, 1 \leq i, j \leq n$ is independent of any $\mathcal{T}_{local}[\ell, q], (\ell, q) \in M, (\ell = i \text{ or } \ell < i - K - 1), 1 \leq q \leq n$.* $\square$

We maintain n Emde Boas data structures $E_i, 1 \leq i \leq n$, one for each column. We also need to maintain one *insert list*, $\mathcal{I}$ and one *delete list*, $\mathcal{D}$. Recall that we proceed in a row by row manner. Suppose we are starting to process row $i + K + 2$ i.e. we are considering the 'first' match in this row, namely, $(i + K + 2, j) \in M$. So we need to calculate $\mathcal{T}_{local}[i + K + 2, j]$ and $\mathcal{T}_{global}[i + K + 2, j]$. At this instant the delete list $\mathcal{D}$ contains all (i, ℓ) such that $1 \leq \ell \leq n, (i, \ell) \in M$ and the insert list $\mathcal{I}$ contains all $(i + K + 1, \ell)$ such that $1 \leq \ell \leq n, (i + K + 1, \ell) \in M$. In other words, when we consider the first match in row $(i + K + 2)$, $\mathcal{D}$ contains all the matches in row i and $\mathcal{I}$ contains all the matches in row $i + K + 1$. For each $(m, n) \in \mathcal{D}$ we delete (m, n) from E_n and for each $(m, n) \in \mathcal{I}$ we insert (m, n) in E_n . Note that the sorted order in E_n is maintained according to the $\mathcal{T}_{local}$ -value. We then calculate $\mathcal{T}_{local}[i + K + 2, j]$ for all $1 \leq j \leq n$ such that $(i + K + 2, j) \in M$. It should be clear that we can calculate $\mathcal{T}_{local}[i + K + 2, j]$ as follows in $O(K)$ time:

$$\mathcal{T}_{local}[i + K + 1, j] = \max_{j - K - 1 \leq \ell \leq j - 1} (\text{value}(\max(E_\ell))) \quad (4)$$

Note that $\text{value}(\max(E_\ell)) = \mathcal{T}_{local}[m, n]$ when $\max(E_\ell) = (m, n)$. The running time of $O(K)$ follows from the fact that we can find the maximum of each E_ℓ in constant time. And the correctness follows from Fact 3.

What should be the running time of this algorithm? It is clear that we spend $O(n^2 + \mathcal{R}K)$ time in computing the LCS for FIG. But this improved time comes at the cost of maintaining n Emde Boas data structure $E_i, 1 \leq i \leq n$. It is easy to verify that the total time to maintain $E_i, 1 \leq i \leq n$ is $O(\mathcal{R} \log \log n)$ because we never insert nor delete more than $\mathcal{R}$ elements in total from/to $E_i, 1 \leq i \leq n$. Note that the values to be inserted is always within the range $[1..n]$ since no subsequence can be of length greater than n .

Theorem 3. *Problem FIG can be solved in $O(n^2 + \mathcal{R}K + \mathcal{R} \log \log n)$ time using $\theta(n^2)$ space.* $\square$

¹ Although we still don't achieve a good running time for Problem LCS in the worst case.

If the solutions for the subproblems are not required, then we need only compute $\mathcal{T}_{local}[i, j]$, $(i, j) \in M$. We, however, would need to use a variable to finally report $r(X, Y)$ and use appropriate pointers to construct $lcs(X, Y)$. Therefore, we get the following theorem (The corresponding algorithm is formally stated in the form of Algorithm 1).

Theorem 4. *Problem FIG can be solved in $O(\mathcal{R}K + \mathcal{R} \log \log n)$ time using $\theta(R)$ space.* $\square$

Algorithm 1.

```

1: Compute the set  $M$  using the preprocessing step suggested in Section 2. Let  $M_i = (i, j) \in M, 1 \leq j \leq n$ .
2: for  $i = j$  to  $n$  do
3:    $E_j = \epsilon$  {Initialize the  $n$  Emde Boas structure one for each column}
4: end for
5:  $\text{globalLCS.Instance} = \epsilon$ 
6:  $\text{globalLCS.Value} = \epsilon$ 
7: for  $i = 1$  to  $n$  do
8:   Insert  $(i - 1, j) \in M_{i-1}$  in  $E_j, 1 \leq j \leq n$  {If  $i - 1 \leq 0$  then insert nothing}
9:   Delete  $(i - K - 2, j) \in M_{i-K-2}$  in  $E_j, 1 \leq j \leq n$  {If  $i - K - 2 \leq 0$  then delete nothing}
10:  for each  $(i, j) \in M_i$  do
11:     $\text{maxresult} = \max_{(j-K-1) \leq \ell \leq (j-1)} (\max(E_\ell))$ 
12:     $\mathcal{T}.\text{Value}[i, j] = \text{maxresult.Value} + 1$ 
13:     $\mathcal{T}.\text{Prev}[i, j] = \text{maxresult.Instance}$ 
14:    if  $\text{globalLCS.Value} < \mathcal{T}.\text{Value}[i, j]$  then
15:       $\text{globalLCS.Value} = \mathcal{T}.\text{Value}[i, j]$ 
16:       $\text{globalLCS.Instance} = (i, j)$ 
17:    end if
18:  end for
19: end for
20: return  $\text{globalLCS}$ 

```

4 A K -Independent Algorithm for FIG

In this section we try to devise an algorithm for FIG that is independent of K . As we shall see later that this would give us an efficient algorithm for Problem LCS as well. We make use of a classical problem in computer science, namely, Range Maxima Query Problem.

Problem “RMAX” (Range Maxima Query Problem). *Given a sequence $A = a_1 a_2 \dots a_n$, a Range Maxima (minima) Query specifies an interval $I = (i_s, i_e), 1 \leq i_s \leq i_e \leq n$ and the goal is to find the index ℓ with maximum (minimum) value a_ℓ for $\ell \in I$.*

Theorem 5. *([4, 3]). The RMAX problem can be solved in $O(n)$ preprocessing time and $O(1)$ time per query.* $\square$

With Theorem 5 in our hand, we can modify Algorithm 1 as follows. We want to implement Step 11 in constant time so that we can avoid the dependency on K completely. Before we start processing a particular row, just after Step 9, we create an array of length n , $A = \max(E_j), 1 \leq j \leq n$. Now we simply

replace the Step 11 with an *appropriate* Range Maxima Query. It is easy to see that, due to Fact 3, this will work correctly. Since we have a constant time implementation for Step 11, we now can escape the dependency on K . However there is a preprocessing time of $O(n)$ in case any E_j gets updated. But since this preprocessing is needed once per row (due to Fact 3), the computational effort added is $O(n^2)$ in total.

Theorem 6. *Problem FIG can be solved in $O(n^2 + \mathcal{R} \log \log n)$ time using $\theta(\max(\mathcal{R}, n))$ space.* $\square$

Finally, it is easy to see that this algorithm can be easily used to solve LCS problem, virtually, without any modification. We, however, can do better using Fact 1 and the subsequent discussion in Section 3. We can get rid of the Emde Boas structures altogether and use a simple array (array H in Section 3) instead. So we get the following theorem.

Theorem 7. *Problem LCS can be solved in $O(n^2 + \mathcal{R} \log \log n)$ time using $\theta(\max(\mathcal{R}, n))$ space.* $\square$

We can shave off the $\log \log n$ term from the running time of Theorem 7 by not computing M as a preprocessing step. In this case, however, we need to process each entry of $\mathcal{T}[i, j]$, $1 \leq i \leq n, 1 \leq j \leq n$, instead of processing only each $(i, j) \in \mathcal{M}$.

Theorem 8. *Problem LCS can be solved in $O(n^2 + \mathcal{R})$ time using $\theta(n^2)$ space.* $\square$

5 Algorithm for Elastic Gapped LCS

In this section we modify the algorithms in Section 2, 3, and 4 to solve ELAG. Recall that, in ELAG, we have two parameters, namely K_1 and K_2 . Note also that, FIG is a special case of ELAG when $K_1 = 0$ and $K_2 = K$. The obvious modification to Equation 2 in Section 2 to solve ELAG is as follows:

$$\mathcal{T}_{local}[i, j] = \begin{cases} \text{Undefined} & \text{if } (i, j) \notin M, \\ \max_{\substack{i-1-K_2 \leq \ell_i < i-K_1 \\ j-1-K_2 \leq \ell_j < j-K_1 \\ (\ell_i, \ell_j) \in M}} (\mathcal{T}_{local}[\ell_i, \ell_j]) + 1 & \text{if } (i, j) \in M. \end{cases} \quad (5)$$

Theorem 9. *Problem ELAG can be solved in $O(n^2 + \mathcal{R}(K+1)^2)$ time using $\theta(n^2)$ space where $K = K_2 - K_1$.* $\square$

For Algorithm 1, described in Section 3, the only modification that is needed to solve ELAG, is in Step 11. The modified statement is as follows:

$$\text{maxresult} = \max_{(j-K_2-1) \leq \ell \leq (j-K_1)} (\max(E_\ell))$$

Theorem 10. *Problem ELAG can be solved in $O(\mathcal{R}K + \mathcal{R} \log \log n)$ time, using $\theta(\max(\mathcal{R}, n))$ space, where $K = K_2 - K_1$.* $\square$

The following result holds if we need the solutions to the subproblems.

Theorem 11. *Problem ELAG can be solved in $O(n^2 + \mathcal{R}K + \mathcal{R} \log \log n)$ time, using $\theta(n^2)$ space, where $K = K_2 - K_1$. $\square$*

Finally, it should be clear that in the algorithm in Section 4, virtually, there is no modification at all except for that we have to adjust the Range Maxima Query to incorporate the elastic gap constraint.

Theorem 12. *Problem ELAG can be solved in $O(n^2 + \mathcal{R} \log \log n)$ time, using $\theta(\max(\mathcal{R}, n))$ space. $\square$*

6 Algorithms for Rigid Gapped LCS

This section is dedicated to solve Problem RIFIG and Problem RELAG. RIFIG, by nature, is a bit more restricted because, in addition to the K -gap constraint, the consecutive characters in the common subsequence must have the same distance between them (rigidness) both in X and Y . Interestingly enough, this restriction makes this problem rather easier to solve. And in fact we will see that we can modify the algorithm in Section 2 easily to solve RIFIG and this slight modification would even improve the running time of the algorithm. The key idea lies in the fact that to calculate a $\mathcal{T}_{local}[i, j]$ we just need to check the $K + 1$ diagonal entries before it. This is true because of the required *rigidness*. The modified version of Equation 2 to handle RIFIG is given below.

$$\mathcal{T}_{local}[i, j] = \begin{cases} \text{Undefined} & \text{if } (i, j) \notin M, \\ \max_{\substack{(\ell_i, \ell_j) \in \{(i-1, j-1), (i-2, j-2), \dots, (i-1-K, j-1-K)\} \\ (\ell_i, \ell_j) \in M}} (\mathcal{T}_{local}[\ell_i, \ell_j]) + 1 & \text{if } (i, j) \in M. \end{cases} \quad (6)$$

Also we can easily modify Equation 6 to solve RELAG. So we get the following theorems.

Theorem 13. *Problem RIFIG can be solved in $O(n^2 + \mathcal{R}K)$ time using $\theta(n^2)$ space. $\square$*

Theorem 14. *Problem RELAG can be solved in $O(n^2 + \mathcal{R}(K_2 - K_1))$ time using $\theta(n^2)$ space. $\square$*

In the rest of this section we will try to achieve better solutions for RIFIG. We first introduce a variant of RIFIG where the gap constraint is withdrawn. In other words we can say that in this variant we have $K = n$.

Problem “RLCS” (Rigid LCS Problem). *Given two strings X and Y , each of length n , we want to find out a Rigid Common Subsequence of the maximum length². A Rigid Common Subsequence of X and Y is a subsequence $S[1..r] = S[1] S[2] \dots S[r]$ of both X and Y such that $C(X, S)[i] - C(X, S)[i-1] = C(Y, S)[i] - C(Y, S)[i-1]$ for all $2 \leq i \leq r$.*

² The generalized version of RLCS was introduced and proved to be Max-SNP hard in [6].

It is easy to see that using Equation 6 we can easily solve Problem RLCS by assuming $K = n$. But this will not give us a very good running time at all. On the other hand, it turns out that, we can achieve a better running time by appropriate modification to Equation 1. To solve RLCS, however, for each tabular entry $\mathcal{T}[i, j]$ we calculate and store two values namely $\mathcal{T}_{local}[i, j]$, $\mathcal{T}_{global}[i, j]$. The recurrence relations are defined below:

$$\mathcal{T}_{local}[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \mathcal{T}_{local}[i - 1, j - 1] + 1 & \text{if } X[i] = Y[j], \\ \mathcal{T}_{local}[i - 1, j - 1] & \text{if } X[i] \neq Y[j]. \end{cases} \quad (7)$$

$$\mathcal{T}_{global}[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1]) & \text{if } (i, j) \notin M, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1], \mathcal{T}_{local}[i, j]) & \text{if } (i, j) \in M. \end{cases} \quad (8)$$

It is easy to see that Equation 7 preserves the rigidness of the subsequence. Note that, Equation 8 is required to keep track of the global solution.

Theorem 15. *Problem RLCS can be solved in $O(n^2)$ time using $\theta(n^2)$ space.* $\square$

Inspired by the idea of above solution to RLCS in the rest of this section we try to devise an algorithm to solve RIFIG in $O(n^2)$ time. The idea is to somehow propagate the *constraint information* up through the diagonal entries as soon as we find a match and whenever a match is found check this information. What we plan to do is as follows. For the calculation of $\mathcal{T}_{local}$ we apply K -modulo arithmetic. The actual length of LCS would be $\lceil \mathcal{T}_{local}[n, n]/K \rceil$.

$$\mathcal{T}_{local}[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ 0 & \text{if } X[i] \neq Y[j] \text{ and} \\ & \mathcal{T}_{local}[i - 1, j - 1] \bmod K = 1, \\ \lceil \mathcal{T}_{local}[i - 1, j - 1]/K \rceil * K + K & \text{if } X[i] = Y[j], \\ \mathcal{T}_{local}[i - 1, j - 1] - 1 & \text{if } X[i] \neq Y[j] \text{ and} \\ & \mathcal{T}_{local}[i - 1, j - 1] > 0, \\ 0 & \text{if } X[i] \neq Y[j] \text{ and} \\ & \mathcal{T}_{local}[i - 1, j - 1] = 0. \end{cases} \quad (9)$$

$$\mathcal{T}_{global}[i, j] = \begin{cases} \mathcal{T}_{global}[i, j] & \text{if } i = 1 \text{ or } j = 1, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1]) & \text{if } (i, j) \notin M, \\ \max(\mathcal{T}_{global}[i - 1, j], \mathcal{T}_{global}[i, j - 1], \mathcal{T}_{local}[i, j]) & \text{if } (i, j) \in M. \end{cases} \quad (10)$$

Theorem 16. *Problem RIFIG can be solved in $O(n^2)$ time using $\theta(n^2)$ space.* $\square$

7 Conclusion

In this paper we have introduced new variants of LCS problem and presented efficient algorithms to solve them. Our algorithms can be used to solve some other interesting problems, specially in bioinformatics, as well. We can tackle the degenerate strings in biological applications, for e.g., using a clever technique invented by Lee et al. [5] (the details are omitted for space constraints). Moreover we can solve the Longest Common Substring problem for degenerate strings, a very common problem in molecular biology simply by putting $K = 1$

in the Problem FIG. Also, our algorithms should be useful in extracting long multiple repeats in DNA sequences as follows. One approach to solve this problem efficiently is to apply ‘lossless’ filters [1, 9] where filters apply a necessary condition that sequences must meet to be part of repeats. One way to improve the computational time and the sensitivity of the filters is to compute Longest Common Subsequences between the ordered sequences of exact k-mers used in the filtering technique. However in the case of the filter, the LCS that needs to be computed has bounded span [8] which, again, can be obtained by applying the gap-constraints in LCS.

References

1. ED’NIMBUS. <http://igm.univ-mlv.fr/peterlon/officiel/ednibus/>.
2. S. F. Altschul, W. Gish, W. Miller, E. W. Meyers, and D. J. Lipman. Basic local alignment search tool. *Journal of Molecular Biology*, 215(3):403–410, 1990.
3. M. A. Bender and M. Farach-Colton. The lca problem revisited. In *Latin American Theoretical Informatics (LATIN)*, pages 88–94, 2000.
4. H. Gabow, J. Bentley, and R. Tarjan. Scaling and related techniques for geometry problems. In *Symposium on the Theory of Computing (STOC)*, pages 135–143, 1984.
5. I. Lee, A. Apostolico, C. S. Iliopoulos, and K. Park. Finding approximate occurrence of a pattern that contains gaps. In *Australasian Workshop on Combinatorial Algorithms (AWOCA)*, pages 89–100, 2003.
6. B. Ma and K. Zhang. On the longest common rigid subsequence problem. In *CPM*, pages 11–20, 2005.
7. W. Pearson and D. Lipman. Improved tools for biological sequence comparison. *Proceedings of National Academy of Science, USA*, 85:2444–2448, 1988.
8. P. Peterlongo. Private communication.
9. P. Peterlongo, N. Pisanti, F. Boyer, and M.-F. Sagot. Lossless filter for finding long multiple approximate repetitions using a new data structure, the bi-factor array. In *SPIRE*, pages 179–190, 2005.
10. P. van Emde Boas. Preserving order in a forest in less than logarithmic time and linear space. *Information Processing Letters*, 6:80–82, 1977.
11. R. A. Wagner and M. J. Fischer. The string-to-string correction problem. *J. ACM*, 21(1):168–173, 1974.